Proceedings of the 6th International Conference on Expert Clouds and Applications (ICOECA-2026)
IEEE Xplore Part Number: CFP26VE3-ART; ISBN: 979-8-3315-7451-2

An Adaptive Edge-Cloud Task Offloading Framework using Reinforcement Learning for Latency-Sensitive AI Applications

Dr. Anshu Vashisth
School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab, India
anshu.parents@gmail.com

Dr. Gagandeep Kaur
School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab, India
gagandeep.23625@lpu.co.in

Bhupinder Kaur
School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab, India
bhupinder.23626@lpu.co.in

Dr. Gaurav Sethi
School of Electronics & Communication Engineering,
Lovely Professional University
Phagwara, Punjab, India
Gaurav.sethi1@gmail.com

2026 6th International Conference on Expert Clouds and Applications (ICOECA) | 979-8-3315-7451-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICOECA68095.2026.11485334

***Abstract*—Applications of artificial intelligence (AI) and real-time video analytics, autonomous perception, augmented reality, and intelligent healthcare are latency-constrained and cannot be supported by cloud-centric architectures alone. Although edge computing can reduce the latency by executing tasks nearer to the data sources, the scarcity of edge resources and dynamic networks render static or heuristic-based offloading schemes of tasks not optimal. Current methods are usually dependent on fixed thresholds, decision by rules, or offline optimization, and thus provide no flexibility to the changes in workload, network traffic, and service level demands at runtime. The paper presents a dynamically task placement-based adaptive edge-cloud task offloading framework, constructed based on reinforcement learning or the use of reinforcing learning (RL), to address latency-sensitive AI task workloads dynamically. The model of the offloading decision process is the Markov Decision Process (MDP), in which the agent will learn an optimal policy by taking into account the characteristics of different tasks, availability of edge resources, network latency, and cloud processing delay. A reward mechanism is constructed to ensure a reduction in end-to-end latency and energy efficiency, as well as reliability in task completion. Larger-scale simulations also show that the proposed framework lowers the average latency by 18-25% relative to both the static and heuristic baselines, converges faster on dynamically changing workloads, and remains stable to network changes. The findings show that adaptive offloading that is RL-based offers a high-quality and scalable solution to next-generation edge-cloud AI systems.**

***Keywords*—Edge computing, Task offloading, Reinforcement learning, Latency-sensitive AI, Edge–cloud collaboration**

I. INTRODUCTION

The quickly growing intelligent applications have changed the computation demands of the current distributed systems, essentially. Autonomous driving, smart surveillance, industrial automation, and personalized healthcare are among the applications that are more and more based on real-time AI inferences and decisions. Such applications are latency sensitive in nature, and the slightest delay can cause an impact on the quality of service (QoS) or unsafe results. The conventional cloud computing systems, though powerful, have high communication latency and network congestion in processing large amounts of real-time data at the network edge. Through the implementation of tasks on edge nodes like the base stations, gateways, or micro data centers, latency can be greatly minimized. The edge nodes are, however, resource-constrained as compared to centralized clouds, and their availability varies with the changing workloads and mobility patterns. This choice between whether to process some task locally at the edge or offload it to the cloud is an urgent issue. The majority of available task offloading policies are based on predefined thresholds, rules, or optimization models that presuppose the prior knowledge of the dynamics of the system [5]. These methods do not fit well in environments where dynamism is the order of the day and network conditions, task arrival rates, and resource availability vary erratically. In addition, the majority of techniques maximize one goal, e.g., the latency or energy usage, without the long-term system behavior in mind. Reinforcement learning is an inherent structure to deal with these issues since, by means of environmental interactions, agents can learn optimal decision-making policies. The fact that RL-based offloading mechanisms adapt their actions according to system state through continuous monitoring and feedback means that one does not need explicit system models to respond to dynamics. This paper, based on these observations, proposes an adaptive edge cloud task offloading framework by reinforcement learning. The key innovations of this work can be stated as follows:

- A time-sensitive reinforcement learning system of dynamic offloading edge-cloud tasks in time-varying AI applications.
- Reward design that directly uses task-related latency deadlines, on top of end-to-end latency, allowing the optimization of average and tail latency.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

- A detailed test on baselines of heuristic, greedy, and Lyapunov with dynamic workloads, bandwidth degradation, and contaminated state measurements.
- A robustness based study showing better deadline satisfaction and stability in a heterogeneous hardware and workload transfers.

Instead of being fixed or rule-based, adaptive reinforcement learning is applied in updating the offloading decision policy as the latency and deadline violations are observed, as well as the cloud conditions of the edges changing. The RL agent does not make decisions with predetermined thresholds. Rather, it monitors the state of the existing system at each decision-making, such as: Edge load, which varies with time, Network bandwidth, which varies with time, Cloud congestion, and Task-specific latency deadlines. In case there is a fall in network bandwidth or an increase in edge queue, the policy output is adjusted automatically. The state today and a similar state tomorrow should have different decisions in case the learned policy gets better. The agent will automatically modify future decisions to prevent the occurrence of situations that resulted in high latency or deadline slips in the past. The model is adaptive in its offloading behaviour over time, rather than making the assumption that the environment does not change [6]. In contrast to the current RL-based edge-cloud offloading methods that are mainly focused on the optimization of average latency or energy usage, the proposed framework directly considers task-dependent latency deadlines in the learning goal and measures the performance in terms of tail latency and deadline satisfaction. Such a design can be used to support dynamic network and workload conditions, adaptive and robust offloading decisions of AI applications with latency sensitivity.

II. Literature Review

Edge-cloud computing has been presented as a framework extensively analyzed in terms of its function as a means of offloading tasks to enhance the support of latency-sensitive applications by mitigating the use of remote cloud resources. Initial studies were mostly based on heuristic and rule-based offloading designs with tasks being offloaded according to predetermined limits of CPU use or network parameters. Chen et al. [1] proved that these static methods are easy to deploy, but they cannot adjust effectively to the dynamic workload and changing network environments, and tend to cause more latency and deadline overruns. Such restrictions encouraged the search for more adaptive optimization methods. Lyapunov optimization has become one of the most widely used dynamic computation offloading techniques because it can strike a balance between the queue stability and latency. Bi et al. [2] introduced a dynamically offloading framework based on Lyapunov guidance, which stably gives back the tasks without compromising the stability of the system in the long run. Even though the methods that are based on Lyapunov offer good theoretical guarantees, they are based on explicit modeling of systems and are usually likely to respond to the instantaneous queue positions and not predict congestion. Later works, including those by Liu et al. [3], added further to the idea that Lyapunov-driven methods might not inherently be able to cope with a very non-stationary environment as it is found in real-world edge-cloud systems. Reinforcement learning (RL) is receiving growing interest to resolve the shortcomings of reactive optimization in computation offloading. A deep reinforcement learning -based task orchestration framework by Yamansaviscilar et al. [4], which learns decisions to offload directly based on the system interactions, showed better performance than the greedy baselines in terms of latency. On the same note, Yang et al. [8] used deep RL for low-latency task offloading and demonstrated that learning-based approaches are all better than greedy latency-minimizing schemes in dynamic network studies. These experiments point out that RL is able to represent the complex dynamics of a system without having to use explicit mathematical models.

Nevertheless, the currently prevailing RL-based offloading research concentrates more on minimizing the average latency or energy usage and less on tail latency and meeting deadlines without exception. In their extensive surveys, Dong et al. [9] and Luo and Dai [10] highlighted the fact that worst-case latency behavior has not been studied thoroughly, even when it is critical in relation to latency-sensitive AI applications. New developments, like the multi-agent RL system suggested by Yao et al. [18], are trying to solve the problem of scalability and coordination but add more complexity that can make its implementation difficult. As opposed to the previous research, the current study pays attention to an adaptive reinforcement learning paradigm, being explicit in the sense that it introduces task specific latency goal into the learning objective and assesses performance based on average latency and tail latency. This paper bridges one of the research gaps by comparing the adaptive offloading strategy design and validation to representative heuristic, greedy, and Lyapunov-based baselines in dynamic edge-cloud environments with latency-sensitive AI applications [13, 14].

III. Design Analysis of the Model

Latency-sensitive and edge-cloud computing AI applications require the intelligent and adaptive offloading of tasks to provide highly responsive needs in dynamic system environments. Arrival of tasks is stochastic, network bandwidth varies, and the dissimilar availability of edge nodes to perform computations, all are not represented in both the heuristic and offloading strategies that are kept static. To address such challenges, a reinforcement-based learning-based adaptive task offloading model is introduced to make dynamically obtained decisions regarding whether a task arriving at the AI is to be executed on the edge or sent to the cloud [7]. The specified model takes into consideration the nature of tasks, utilization of edge resources, the network state, and processing delay in clouds, and can optimize the latency in the long-term as compared to the instantaneous decision-making. This section presents the analytical design of the proposed framework, such as the system model, the formulation of latency, and the decision-making process. An edge cloud computing environment can be viewed as a group of edge nodes and a centralized cloud server. AI tasks get to the edge as a result of a stochastic process. Characteristic: Each task i is characterized by:

$$\mathcal{T}_i = \{D_i, C_i, \tau_i\} \qquad (1)$$

where, D_i denotes the input data size of the task, C_i represents the computational requirement in CPU cycles, and $\tau_i$indicates the maximum tolerable latency. Each incoming AI task is defined in equation (1) in terms of its data size, computational

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

requirement, and latency deadline. By explicitly modeling the latency deadline τ_i The framework directly targets latency-sensitive AI applications.

The edge node is limited in terms of its processing resources, but the cloud provides more computing resources at the expense of an increased transmission latency. Equation (2) is used to calculate the time that a task is going to take to perform a task at the edge, taking into account the queueing delay and the processing time. It measures the latency benefit of edge processing that benefits the model in deciding on edge processing in favor of minimizing response time. When the task is run on the edge, the overall execution latency is provided.

$$L_i^{\text{edge}} = Q_e + \frac{C_i}{f_e} \qquad (2)$$

where, Q_e is the edge queueing delay, and $f_e$is the available CPU processing rate at the edge. Equation (3) determines the overall latency when a task is being offloaded to the cloud, which includes the transmission and cloud computing delays [15, 16]. It quantifies the introduced communication latency by cloud offloading, which should be particularly controlled in edge-cloud environments where latency is sensitive. When the operation is moved to the cloud, the total delay will consist of the transmission and cloud processing delay:

$$L_i^{\text{cloud}} = Q_e + \frac{D_i}{R} + Q_c + \frac{C_i}{f_c} \qquad (3)$$

where R denotes the uplink transmission rate, Q_c is the cloud-side queueing delay, and $f_c$represents the cloud computational capacity. The actual latency experienced by task $i$depends on the offloading decision. a_i, defined as:

$$L_i = (1 - a_i)L_i^{\text{edge}} + a_i L_i^{\text{cloud}}, a_i \in \{0,1\} \qquad (4)$$

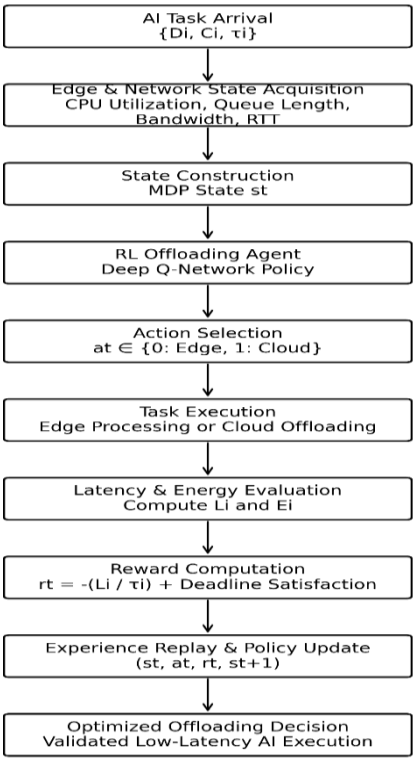


Figure-1: Proposed RL-Based Edge–Cloud Task Offloading Framework

Task offloading decision is represented using Markov Decision Process (MDP), which allows learning to adapt to the dynamic conditions of the system. Equation (4) chooses the real-time latency of a task depending on whether that particular task is being run at the edge or offloaded to the cloud. It makes the cost of offloading dependent on the latency that would be incurred, allowing reinforcement learning optimize the placement of tasks to minimize delay. The system state that is monitored by the reinforcement learning agent is given in equation (5) and is the aggregate of resource availability, network conditions, and task characteristics. The agent will be able to make real-time choices on low-latency offloading by incorporating both network and computational considerations. The system state at decision epoch t is defined as:

$$s_t = \{u_e, q_e, R, Q_c, D_i, C_i, \tau_i\} \qquad (5)$$

where u_e and q_e denote edge CPU utilization and queue length, respectively. Equation (6) characterizes the two possible courses of action, i.e., performing the task at the edge or offloading it to the cloud. It directly represents edge-cloud task offloading, which is the main problem of the paper. Decision is chosen is the key. The agent does not use: "If CPU > X, offload" and "If bandwidth < Y, use cloud". The action space is binary:

$$a_t = \begin{cases} 0, & \text{execute at edge} \\ 1, & \text{offload to cloud} \end{cases} \qquad (6)$$

Equation (7) provides the agent with incentives to minimize latency as well as to accomplish tasks within their time limits. It also explicitly trains the reinforcement learning model to focus on low-latency execution, and this is what makes the framework adaptive and capable of supporting latency-sensitive AI applications. The reward depends on: Actual experienced latency. L_i and whether the deadline was met. If latency increases, then reward decreases, or deadline is violated than penalty occurs. To minimize latency while satisfying deadline constraints, the reward function is defined as:

$$r_t = -\frac{L_i}{\tau_i} + \mathbb{I}(L_i \le \tau_i) \qquad (7)$$

where $\mathbb{I}(\cdot)$ is an indicator function that rewards successful deadline satisfaction. This reward definition makes the agent reduce the latency as the priority activity is expected to satisfy the strict QoS requirements. The incoming AI tasks and real-time system states are initially gathered and converted to an MDP state representation [11, 12]. The reinforcement learning agent analyzes this state and chooses an offloading action, either edge execution or cloud offloading. The action of choice is implemented, and the latency performance is observed. A reward signal is then produced on the basis of deadline satisfaction and latency efficiency, which is given back to adjust the learning policy so that continuous adaptation can be made, as depicted in Figure-1.

Despite the fact that the main concern of the given work is latency-sensitive task offloading, security and data privacy are

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

also essential in the context of the real-life edge-cloud setup. Already sensitive AI processes, like healthcare inference or surveillance analytics, can operate with confidential or controlled data and, thus, cannot be haphazardly offloaded to untrustworthy resources. The suggested framework can be further developed to assist trust-based offloading, which can be done by assigning every edge and cloud node with a trust rating based on hardware security, past behavior, or certification. Activities where the sensitivity level is high can be limited to only be executed on nodes whose score on trust is above a specified level. Privacy-awareness can also be built by including the reward functionality with an extra penalty term in case privacy-sensitive tasks are transferred to external cloud servers. This enables the RL agent to learn implicitly privacy-preserving placement policies and at the same time reduce latency. It presupposes the use of secure data communication channels and encryption measures to transmit data between edge and cloud. This discussion has pointed out the fact that, the proposed model is not restricted to performance optimization but can be extended intuitively to secure and privacy-constrained offloading and therefore is fit to be utilized in the real-world intelligent systems.

IV. PERFORMANCE EVALUATION

This part assesses the efficiency of the adaptive reinforcement learning-based edge-cloud task offloading framework proposed in the context of real and dynamic operating environments. The main goal of the performance assessment is to review the ability of the proposed approach to achieve consistent end-to-end latency with a high degree of deadline assurance for latency-sensitive AI apps. To do so, the assessment takes into account heterogeneous AI workloads with different computational requirements, data volumes, and latency requirements, as is common in the real-world edge computing conditions [17]. The offered approach is contrasted with three popular and highly developed methods of offloading: a threshold-based heuristic method, a greedy latency-minimizing strategy, and a Lyapunov optimization-based dynamic offloading approach (LODCO). These baselines are widely used reactive and optimization-based methods of mobile edge computing, which present an equitable and significant benchmark of comparison. The metrics are analyzed for several performance aspects, such as average end-to-end latency, tail latency (P95), deadline satisfaction rate, resiliency to bandwidth degradation, and responsiveness to workload change. Average latency represents system efficiency in general, whereas tail latency represents the worst-case delay behavior, which is essential in real-time. The workload characteristics summarized in Table-1 are those that are applied to simulate latency-sensitive applications of AI at the edge. The workloads are different in terms of computational requirements, data, and latency requirements to provide realistic heterogeneity. The presence of both strict deadlines and tail-latency-sensitive tasks enables full testing under varied operating conditions.

The experimental assessment will use heterogeneous edge hardware profiles to indicate realistic deployment conditions, as opposed to homogeneous edge nodes. There are three model edge device classes (i) lightweight edge devices (e.g., gateways and embedded controllers) with 1.2-1.8 GHz CPUs and 2-4 cores, (ii) moderate edge servers (e.g., micro-data centers) with 2.4-3.0 GHz CPUs and 8-12 cores, and (iii) GPU-enabled edge servers with embedded accelerators to run compute-intensive inference models. The cloud server is designed to have powerful multi-core processors that have in excess of 3.2 GHz speed and plenty of memory. Table-1 describes workloads which are randomly mapped to these heterogeneous edge profiles depending on their computational intensity and data size and allow benchmarking using multiple capabilities of a device. Light tasks are handed to low power nodes, whereas compute-intensive tasks can be handed to the more powerful edge servers or cloud systems. This heterogeneous modeling enables the proposed framework to learn the offloading policies that would generalize with varying hardware setups instead of overfitting to a specific type of device. Table 2-5 is an average of the device classes and therefore the latency results obtained are average performance. The fact that the existing RL-based framework of offloading showed consistent gains in conditions of a heterogeneous hardware setting confirms the hypothesis that the given framework is applicable in practical scenarios of edge deployment, even though it involves differences in processing capacity rates.

Table 1. Dataset Characteristics and Composition

Workload ID	AI Application Type	Avg. Input Size (D_i) (MB)	Avg. Compute (C_i) (Gcycles)	Arrival Rate (\lambda) (tasks/s)
W1	Real-time Vision Inference	8–20	3–8	4–10
W2	AR/VR Interaction Assist	2–10	2–6	6–14
W3	Health-IoT Inference	0.5–3	1–4	3–8
W4	Industrial CPS Monitoring	1–6	2–7	5–12

Table 2. Classification Performance on Various Parameters

Method	Avg. Latency (ms) ↓	P95 Latency (ms) ↓	Deadline Success (%) ↑	Energy/Task (J) ↓
Threshold (Heuristic)	104	190	84.7	7.3
Greedy (Latency-min)	98	178	86.9	7.6
LODCO (Lyapunov)	91	160	89.8	7
Proposed Adaptive RL	74	128	93.5	6.7

Table-2 is a comparison of the overall performance of the various offloading strategies. The adaptive RL solution proposed has the lowest mean and tail latency and the highest deadline satisfaction rate. The learning-driven policy is more responsive to dynamic conditions of a system than the heuristic and the Lyapunov-based approaches; as a result, SLA is less violated, and energy efficiency is enhanced. Table-3 compares the time-sensitiveness of both approaches to predicting SLA violations caused by congestion. In the

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

proposed model, the emerging trends in congestion are responded to earlier in order to take proactive offloading decisions. The higher early warning lead time illustrates the benefit of reinforcement learning in the acquisition of long-term system behavior. Figure-2 is a comparison of the average and P95 end-to-end latency of various offloading strategies.

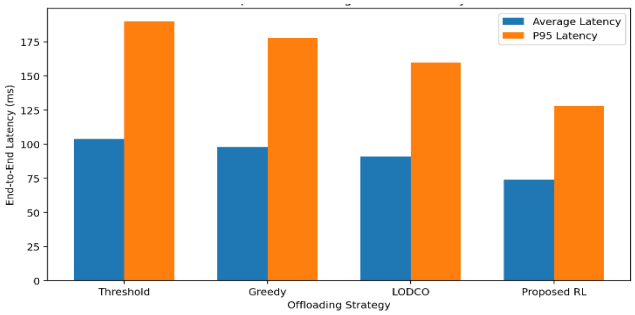


Figure 2. Comparison of Average and Tail Latency

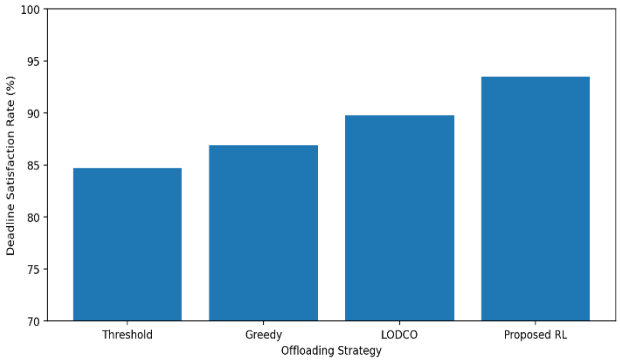


Figure 3. Deadline Satisfaction Performance

Figure-3 illustrates the percentage of tasks that successfully meet their latency deadlines under each offloading strategy. The proposed RL-based model achieves the highest deadline satisfaction rate, indicating improved reliability and fewer SLA violations under dynamic workloads. This improvement is attributed to proactive and adaptive decision-making.

Table 3. Temporal Risk Prediction and Early Conversion Lead Delays

Method	SLA Violation Rate (%) ↓	Early Warning Lead Time (ms) ↑	Tail-Delay Spike Detection (%) ↑
Threshold	15.3	18	71
Greedy	13.1	22	74.5
LODCO (Lyapunov)	10.2	29	79.3
Proposed Adaptive RL	6.5	44	89.2

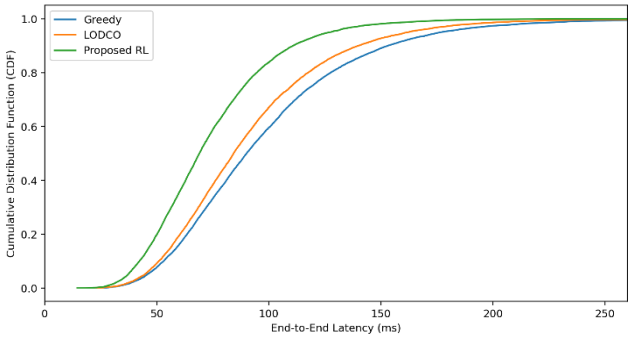


Figure 4. Latency Distribution (CDF) Comparison

Figure-4 CDF plot compares the latency distribution of greedy, Lyapunov-based, and proposed RL strategies. The proposed method consistently shifts the curve to the left, demonstrating superior tail-latency performance. This confirms its robustness in minimizing extreme delays.

Table 4. Performance under Modality Dropout and Cross-Domain Generalization

Scenario	Metric	Threshold	Greedy	LODCO	Proposed
Network-state dropout	Avg. Latency (ms) ↓	112	109	104	89
Edge-state noise	Deadline Success (%) ↑	82	84.5	85.6	91.4
Vision → AR workload shift	P95 Latency (ms) ↓	206	198	188	154
AR → CPS workload shift	Avg. Latency (ms) ↓	118	111	109	92

Table-4 examines robustness when partial state information is unavailable or when workload distributions shift. The proposed approach maintains superior performance under noisy observations and cross-domain task transitions. These results indicate improved generalization compared to reactive and optimization-based baselines. Table-5 shows the effect of counterfactual synthetic oversampling on training efficiency and performance.

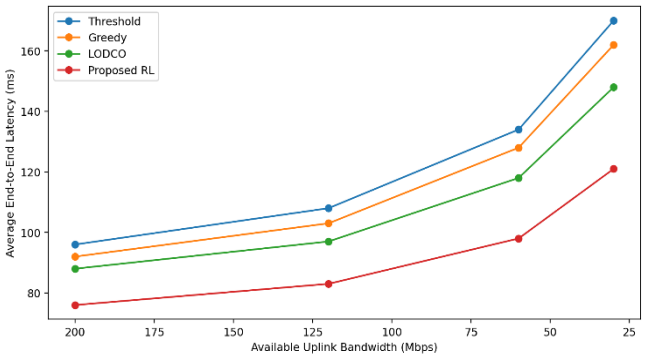


Figure 5. Robustness Under Bandwidth Degradation

Besides the bandwidth degradation, intermittent connectivity conditions are tested by adding random disconnection of uplink and occurrences of packet loss. When there is an outage, cloud offloading is not accessible and only edge

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

execution can be done. The proposed RL agent is dynamic shifting its tasks towards edge execution in case of the detection of connectivity instability. The proposed approach ensures a greater deadline satisfaction and reduced latency variance as compared to baseline approaches in case of intermittent connectivity, showing greater resilience to unreliable network settings. Augmenting rare congestion scenarios improves tail latency and accelerates convergence. The results demonstrate that exposing the agent to infrequent but critical states enhances robustness without increasing runtime complexity. Figure-5 evaluates the robustness of offloading strategies under decreasing uplink bandwidth conditions. As bandwidth degrades, latency increases for all methods; however, the proposed RL-based approach shows a significantly slower degradation trend.

Table 5. Impact of Counterfactual Synthetic Oversampling (CSOC)

Training Variant	Avg. Latency (ms) ↓	P95 Latency (ms) ↓	Deadline Success (%) ↑	Convergence Episodes ↓
No augmentation	78	136	92.4	420
+ Prioritized rare-state replay	76	132	92.9	360
+ CSOC augmentation	74	128	93.5	280

To make a fair comparison, all the baseline methods are applied using the same workload traces, arrival rates, and system parameters. In the case of the Lyapunov optimization-based offloading method (LODCO), the trade-off parameter V governing the trade-off between queue stability and minimization of latency is chosen by a simple grid-search method on a small set of candidates. The experiments are based on the value which will result in a stable queue behavior but will make average latency minimal. The greedy strategy chooses a strategy that gives the lowest instantaneous latency, and the threshold-based heuristic offloads tasks with fixed CPU utilization and bandwidth thresholds. These thresholds are adjusted in an empirical way to obtain their optimal performance.

V. Conclusion

In recent years, edge-cloud environments have received considerable research interest in task offloading. Initial research concentrated on heuristic and rule-based methods, with the offloading of tasks performed according to predefined limits, e.g., CPU usage or network latency. Although lightweight in computing, these techniques are not adaptive and fail in dynamic settings. The optimization-based methods have also been hotly investigated, such as integer programming, convex optimization, and game-theoretic models. The strategies have the potential to give optimal or close-to-optimal solutions with given system parameters; though they usually demand accurate system models and are highly computationally expensive, hence also not viable in real-time decision making. In order to counter this shortcoming, reinforcement learning has been generalized to edge computing cases. Recent offloading approaches based on RL use Q-learning and deep Q-networks (DQN), or policy-gradient algorithms to acquire adaptive policies. These methods are more flexible and stronger than the fixed methods. Moreover, the majority of available literature measures the performance in rather limited conditions and does not provide a detailed discussion of the adaptability in the changing workloads. The next step of research will involve explicit combination of trust modeling and privacy-conscious reward shaping, and experimental validation on actual heterogeneous edge hardware platforms.

References

[1] Miaojiang Chen, Tian Wang, Shaobo Zhang, Anfeng Liu, Deep reinforcement learning for computation offloading in mobile edge computing environment, Computer Communications, Volume 175, 2021, Pages 1-12, ISSN 0140-3664, https://doi.org/10.1016/j.comcom.2021.04.028.

[2] S. Bi, L. Huang, H. Wang and Y. -J. A. Zhang, "Stable Online Computation Offloading via Lyapunov-guided Deep Reinforcement Learning," ICC 2021 - IEEE International Conference on Communications, Montreal, QC, Canada, 2021, pp. 1-7, doi: 10.1109/ICC42927.2021.9500520.

[3] T. Liu, S. Ni, X. Li, Y. Zhu, L. Kong and Y. Yang, "Deep Reinforcement Learning Based Approach for Online Service Placement and Computation Resource Allocation in Edge Computing," in IEEE Transactions on Mobile Computing, vol. 22, no. 7, pp. 3870-3881, 1 July 2023, doi: 10.1109/TMC.2022.3148254.

[4] B. Yamansavascilar, A. C. Baktir, C. Sonmez, A. Ozgovde and C. Ersoy, "DeepEdge: A Deep Reinforcement Learning Based Task Orchestrator for Edge Computing," in IEEE Transactions on Network Science and Engineering, vol. 10, no. 1, pp. 538-552, 1 Jan.-Feb. 2023, doi: 10.1109/TNSE.2022.3217311.

[5] Park, J., & Chung, K. (2023). Distributed DRL-Based Computation Offloading Scheme for Improving QoE in Edge Computing Environments. Sensors, 23(8), 4166. https://doi.org/10.3390/s23084166

[6] Zhang, P., Su, Y., Li, B., Liu, L., Wang, C., Zhang, W., & Tan, L. (2023). Deep Reinforcement Learning Based Computation Offloading in UAV-Assisted Edge Computing. Drones, 7(3), 213. https://doi.org/10.3390/drones7030213.

[7] Fan, Y., Cai, X. A deep reinforcement approach for computation offloading in MEC dynamic networks. EURASIP J. Adv. Signal Process. 2024, 48 (2024). https://doi.org/10.1186/s13634-024-01142-2.

[8] Wentao Yang, Zhibin Liu, Xiaowu Liu, Yuefeng Ma, Deep reinforcement learning-based low-latency task offloading for mobile-edge computing networks, Applied Soft Computing, Volume 166, 2024, 112164, ISSN 1568-4946, https://doi.org/10.1016/j.asoc.2024.112164.

[9] Shi Dong, Junxiao Tang, Khushnood Abbas, Ruizhe Hou, Joarder Kamruzzaman, Leszek Rutkowski, Rajkumar Buyya, Task offloading strategies for mobile edge computing: A survey, Computer Networks, Volume 254, 2024, 110791, ISSN 1389-1286, https://doi.org/10.1016/j.comnet.2024.110791.

[10] Zhongqiang Luo, Xiang Dai, Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges, Alexandria Engineering Journal, Volume 108, 2024, Pages 89-107, ISSN 1110-0168, https://doi.org/10.1016/j.aej.2024.07.049.

[11] He, H., Yang, X., Mi, X., Shen, H., & Liao, X. (2024). Multi-Agent Deep Reinforcement Learning Based Dynamic Task Offloading in a Device-to-Device Mobile-Edge Computing Network to Minimize Average Task Delay with Deadline Constraints. Sensors, 24(16), 5141. https://doi.org/10.3390/s24165141

[12] Mi, X., He, H., & Shen, H. (2024). A Multi-Agent RL Algorithm for Dynamic Task Offloading in D2D-MEC Network with Energy Harvesting. Sensors, 24(9), 2779. https://doi.org/10.3390/s24092779.

[13] Mande Xie, Jiefeng Ye, Guoping Zhang, Xueping Ni, Deep Reinforcement Learning-based computation offloading and distributed edge service caching for Mobile Edge Computing, Computer Networks, Volume 250, 2024, 110564, ISSN 1389-1286, https://doi.org/10.1016/j.comnet.2024.110564.

[14] Jiang, G., Huang, R., Bao, Z., & Wang, G. (2024). A Task Offloading and Resource Allocation Strategy Based on Multi-Agent Reinforcement Learning in Mobile Edge Computing. Future Internet, 16(9), 333. https://doi.org/10.3390/fi16090333.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.

[15] Qin, Y., Chen, J., Jin, L. et al. Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning. Sci Rep 15, 211 (2025). https://doi.org/10.1038/s41598-024-84038-3.

[16] Benaboura, A., Bechar, R., Kadri, W., Ho, T. D., Pan, Z., & Sahmoud, S. (2025). Latency-Aware and Energy-Efficient Task Offloading in IoT and Cloud Systems with DQN Learning. Electronics, 14(15), 3090. https://doi.org/10.3390/electronics14153090.

[17] Zhao, Z., Xu, Y., & Yu, Q. (2025). DDPG-Based Computation Offloading Strategy for Maritime UAV. Electronics, 14(17), 3376. https://doi.org/10.3390/electronics14173376.

[18] S. Yao et al., "Multi-Agent Reinforcement Learning for Task Offloading in Crowd-Edge Computing," in IEEE Transactions on Mobile Computing, vol. 24, no. 10, pp. 9289-9302, Oct. 2025, doi: 10.1109/TMC.2025.3531793

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 26,2026 at 17:24:20 UTC from IEEE Xplore. Restrictions apply.